

计算思维与人工智能

实验六 输入输出与文件操作

TAKE YOU ON A TRIP
GO!

哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

探索 从未停止

实验目的

- 理解基本数据的输入输出过程，掌握常用格式化显示与交互方法。
- 掌握文本文件的读写流程，理解基于文件的简单数据存储与更新。

字符输入输出（快速回顾）

➢ 输入的本质：字符流

- 程序读到的所有内容都是由“字符”组成，包括空格、回车等
- 字符在内存中用 ASCII 编码，如 'A' → 65

➢ 基本字符输入输出

- **getchar()**：从标准输入读取一个字符
- **putchar()**：向屏幕输出一个字符

➢ 常用转义字符

\n —— 换行　　**\t** —— 制表
**** —— 反斜杠　　**\"** —— 双引号

```c
#include <stdio.h>

int main(void) {
    int ch;

    printf("请输入一个字符：");
    ch = getchar();

    printf("您输入的是：");
    putchar(ch);
    printf("\nASCII 码为：%d\n", ch);

    return 0;
}
```

格式化输入输出（快速回顾）

printf：排版与输出

- 使用**格式化说明符**控制输出变量，如 %d、%s
- 可控制宽度、精度实现表格对齐
- **示例**：%-10s 左对齐，%8.2f 宽度8，小数2位

scanf：格式解析与输入

- 从字符流中解析数据，必须传入变量地址
- 读整数会跳过空白，但 %c 不会跳过空白

```
int a, b;
scanf("%d%*c%d", &a, &b);
```

→ 可解析 12+34，结果：a=12，b=34，+被丢弃

%*c 读取一个字符，但不保存

```
#include <stdio.h>

int main(void) {
    char  name[20];
    int   age;
    float score;

    printf("请输入 姓名 年龄 成绩: ");
    scanf("%19s %d %f", name, &age,
&score);

    printf("\n%-19s%4s%8s\n", "姓名", "
年龄", "成绩");
    printf("-------------------------
\n");
    printf("%-19s%4d%8.1f\n", name, age,
           score);

    return 0;
}
```

输入时限制读取长度：安全写法，最多读取 19 个非空白字符，并自动在末尾补上 '\0'

格式化输入输出（快速回顾）

printf：排版与输出

- 使用**格式化说明符**控制输出变量，如 %d、%s
- 可控制宽度、精度实现表格对齐
- **示例**：%-10s 左对齐，%8.2f 宽度8，小数2位

scanf：格式解析与输入

- 从字符流中解析数据，必须传入变量地址
- 读整数会跳过空白，但 %c 不会跳过空白

```
请输入 姓名 年龄 成绩：张三 19 90

姓名                    年龄    成绩
------------------------------------
张三                      19    90.0
```

```c
#include <stdio.h>

int main(void) {
    char  name[20];
    int   age;
    float score;

    printf("请输入 姓名 年龄 成绩：");
    scanf("%19s %d %f", name, &age,
&score);

    printf("\n%-19s%4s%8s\n", "姓名", "
年龄", "成绩");
    printf("-------------------------
\n");
    printf("%-19s%4d%8.1f\n", name, age,
    score);

    return 0;
}
```

输出时控制显示宽度：
左对齐，至少 19 列宽，不足补空格，不限制最大列数

格式化输入输出（快速回顾）

printf：排版与输出

- 使用**格式化说明符**控制输出变量，如 %d、%s
- 可控制宽度、精度实现表格对齐
- **示例**：%-10s 左对齐，%8.2f 宽度8，小数2位

scanf：格式解析与输入

- 从字符流中解析数据，必须传入变量地址
- 读整数会跳过空白，但 %c 不会跳过空白

```
请输入 姓名 年龄 成绩：张三 19 90

姓名                    年龄    成绩
------------------------------------
张三                      19    90.0
```

```c
#include <stdio.h>

int main(void) {
    char  name[20];
    int   age;
    float score;

    printf("请输入 姓名 年龄 成绩：");
    scanf("%19s %d %f", name, &age,
&score);

    printf("\n%-19s%4s%8s\n", "姓名", "
年龄", "成绩");
    printf("-------------------------
\n");
    printf("%-19s%4d%8.1f\n", name, age,
    score);

    return 0;
}
```

按浮点数格式输出，输出时最小宽度为 8 列，保留 1 位小数

文件基础：文本文件 vs 二进制文件

文本文件

- 使用可见字符保存，把数字 65 当作字符串"65"写入文件
- 可用记事本等直接查看内容
- 适合同步展示与手工修改

文本文件示例

```
Alice 90 95 92.5
Bob 78 82 80.0
Cindy 88 91 89.5
```

二进制文件

- 直接保存内存中的二进制数据，十进制 65 转换为二进制0100 0001存储 （十六进制表示为0x41）
- 更高效、更准确，但不可直接阅读
- 适合结构体数组等大批量数据

二进制文件示例

```
41 6C 69 63 65 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 5A 00 00 00 5F 00 00 00 00 00 B9 42
42 6F 62 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 4E 00 00 00 52 00 00 00 00 00 A0 42
43 69 6E 64 79 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 58 00 00 00 5B 00 00 00 00 00 B3 42
```

文件基础： fopen / fclose与文件模式

FILE* 与文件模式

- FILE 结构体封装了与文件操作相关的底层信息
- **fopen()** 用于打开文件，成功时返回指向FILE的指针
- **fclose()** 用于关闭文件，释放相关资源
- 打开文件时必须检查 fopen() 的返回值是否为 NULL

模式	含义
"r"	以**只读**方式打开文本文件
"w"	以**只写**方式打开文本文件
"a"	以**追加**方式写文本文件
"rb"	以**只读**方式打开二进制文件
"wb"	以**只写**方式打开二进制文件
"ab"	以**追加**方式写二进制文件

```c
#include <stdio.h>

int main(void) {
    int n = 65;
    FILE *fp_txt, *fp_bin;
    //打开文件
    fp_txt = fopen("num.txt", "w");
    if (!fp_txt) {
        printf("无法打开 num.txt! \n");
        return 1;
    }
    fprintf(fp_txt, "%d\n", n);
    fclose(fp_txt);   //关闭文件

    fp_bin = fopen("num.bin", "wb");
    if (!fp_bin) {
        printf("无法打开 num.bin! \n");
        return 1;
    }
    fwrite(&n, sizeof(int), 1, fp_bin);
    fclose(fp_bin);

    return 0;
}
```

文件基础： 文件读写方式

操作类型	常用函数	做什么	输入	输出
逐字符 I/O	int **fgetc**(FILE *fp)	从文件读取 1 个字节（字符）	① fp：文件指针	读取的字符（int）或 EOF
	int **fputc**(int ch, FILE *fp)	向文件写入 1 个字节（字符）	① ch：要写入的字符 ② fp：文件指针	写入的字符（int）或 EOF
文本格式 I/O	int **fprintf**(FILE *fp, const char *fmt, ...)	按格式把变量转换成文本并写入文件	① fp：文件指针 ② fmt：格式串 ③ ...：多个变量	成功写入的字符数
	int **fscanf**(FILE *fp, const char *fmt, ...)	从文件读文本并按格式解析为变量	① fp：文件指针 ② fmt：格式串 ③ ...：接收变量的地址	成功读取的字段数
二进制块 I/O	size_t **fwrite**(const void *ptr, size_t size, size_t count, FILE *fp)	将一段连续内存区域（size×count 字节）原样写入文件	① ptr：内存起始地址 ② size：单个元素字节数 ③ count：元素数量 ④ fp：文件指针	成功写入的元素数量
	size_t **fread**(void *ptr, size_t size, size_t count, FILE *fp)	从文件读取 size×count 字节填入内存	① ptr：接收数据的内存地址 ② size：单个元素字节数 ③ count：元素数量 ④ fp：文件指针	成功读取的元素数量

文件基础： 文件读写方式

文件读写方式

操作类型	常用函数	做什么
逐字符 I/O	int **fgetc**(FILE *fp)	从文件读取 1 个字节（字符）
	int **fputc**(int ch, FILE *fp)	向文件写入 1 个字节（字符）

➢ **返回值**

- **成功**：返回读取或写入的字符，以 int 类型表示（值范围 0 ~ 255，对应 unsigned char）
- **失败**：返回 EOF（通常定义为 -1）

```
char *text = “Hello. This is a demo.\n";

for (int i = 0; text[i] != '\0'; i++) {
    // 逐字符写入
    fputc(text[i], fp);
}
……
int ch, count = 0;
// 逐字符读出
while ((ch = fgetc(fp)) != EOF) {
    count++;
}
……
```

文件基础： 文件读写方式

文件读写方式

操作类型	常用函数	做什么
文本格式 I/O	int **fprintf**(FILE *fp, const char *fmt, ...)	按格式把变量转换成文本并写入文件
	int **fscanf**(FILE *fp, const char *fmt, ...)	从文件读文本并按格式解析为变量

➢ **返回值**

- **成功：** fsanf()　返回成功转换并赋值的项数
  fprintf() 返回实际写入的字符数
- **失败：** 返回 EOF（通常定义为 -1）

```
Student stu[2] = {
    {"Alice", 90, 95, 92.5},
    {"Bob",   78, 82, 80.0},
};
fp = fopen("score.txt", "w"); //写方式打开文件
...
for (int i = 0; i < 2; i++) {
    fprintf(fp, "%s %d %d %.1f\n",
            stu[i].name, stu[i].s1,
            stu[i].s2, stu[i].avg);//写文件
}
fclose(fp);

fp = fopen("score.txt", "r"); //只读方式打开文件
Student t[2];
int i = 0;
...
while (fscanf(fp, "%19s %d %d %f",
              t[i].name, &t[i].s1, &t[i].s2,
              &t[i].avg) == 4) {...}//读文件
fclose(fp);
```

文件基础： 文件读写方式

文件读写方式

操作类型	常用函数	做什么
二进制块 I/O	size_t **fwrite**(const void *ptr, size_t size, size_t count, FILE *fp)	将一段连续内存区域（size×count 字节）原样写入文件
	size_t **fread**(void *ptr, size_t size, size_t count, FILE *fp)	从文件读取 size×count 字节填入内存

➢ **返回值**

- **成功：** fwrite() 返回实际写入的数据项个
  fread() 返回实际读取的数据项个数
- **失败：** 返回值 ≤ count

```
fp = fopen("score.bin", "wb");
...
//写入2个结构体
fwrite(stu, sizeof(Student), 2, fp);
fclose(fp);

fp = fopen("score.bin", "rb");
Student r[2];

//读取2个结构体
fread(r, sizeof(Student), 2, fp);

for (int i = 0; i < 2; i++) {
    printf("%-10s %4d %4d %6.1f\n",
           r[i].name, r[i].s1,
           r[i].s2, r[i].avg);
}
fclose(fp);
```

常见错误与调试建议

输入相关错误

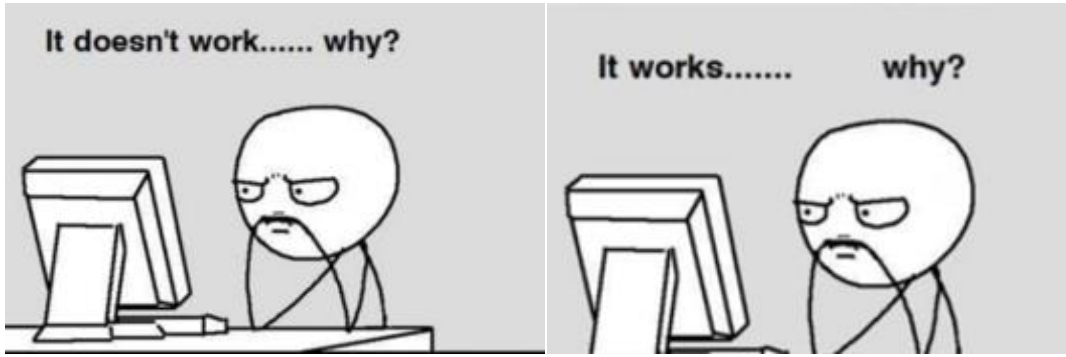
- scanf 忘记写 &
- %c 读取到回车
- %s 不能读含空格的姓名

文件相关错误

- fopen 失败，未检查 fp == NULL
- 文本/二进制模式混用
- feof 用法错误导致多读或死循环

调试建议

- 多使用 printf 查看变量值
- 先在小范围测试，再扩展
- 注意编译器的警告信息



输入输出与文件操作

6001 反应速度游戏

- **题目：**

  在一个**反应速度游戏**中，每次游戏会得到一个整数 **T（毫秒）** 表示玩家从看到提示到按下 Enter 的用时。为了方便后续分析，希望将每局游戏的成绩 **记录到成绩文件 score.txt 中**，并能在屏幕上查看所有历史成绩。

请你编写程序，完成以下功能的核心部分：

① **从成绩文件（文本格式）中读取已有记录**，加载到内存中（结构体数组）；

② 玩一局游戏后，得到本次的 **反应时间 T**，从键盘输入玩家的 **姓名 name**；

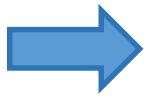
③ 将一条新记录 name T 当前时间 **追加** 到成绩文件末尾，同时更新内存中的数组；

④ **按表格形式输出所有成绩记录**，每行显示：姓名、反应时间（毫秒）、记录时间（YYYY-MM-DD HH:MM:SS，如Alice 320 2025-11-29 14:03:59）。

实验内容

初始文件：
score.txt

```
Bob 310 2025-11-29 09:15:03
Cindy 450 2025-11-29 20:01:55
```



```
Bob 310 2025-11-29 09:15:03
Cindy 450 2025-11-29 20:01:55
Alice 285 2025-11-29 14:03:59
```

【输入输出示例】

```
已从 score.txt 读取 2 条记录。
---------- 反应速度测试 ----------
1. 开始游戏
2. 查看成绩
0. 退出
请选择：1

（随机等待...）

现在按 Enter!
你的反应时间是：285 毫秒。
是否保存本次成绩？(y/n)：y
请输入姓名（不含空格）：Alice
成绩已保存到 score.txt。
```

```
---------- 反应速度测试 ----------
1. 开始游戏
2. 查看成绩
0. 退出
请选择：2

编号  姓名      反应时间(ms)   记录时间
-----------------------------------------------------
1     Bob       310            2025-11-29 09:15:03
2     Cindy     450            2025-11-29 20:01:55
3     Alice     285            2025-11-29 14:03:59

---------- 反应速度测试 ----------
1. 开始游戏
2. 查看成绩
0. 退出
请选择：0
```

6001 反应速度游戏

- **定义结构体**

成员变量	类型	含义
name	char[20]	玩家姓名
ms	int	反应时间，单位：毫秒
date	char[11]	保存“YYYY-MM-DD”格式日期
time_str	char[9]	保存“HH:MM:SS”格式时间

输入输出与文件操作

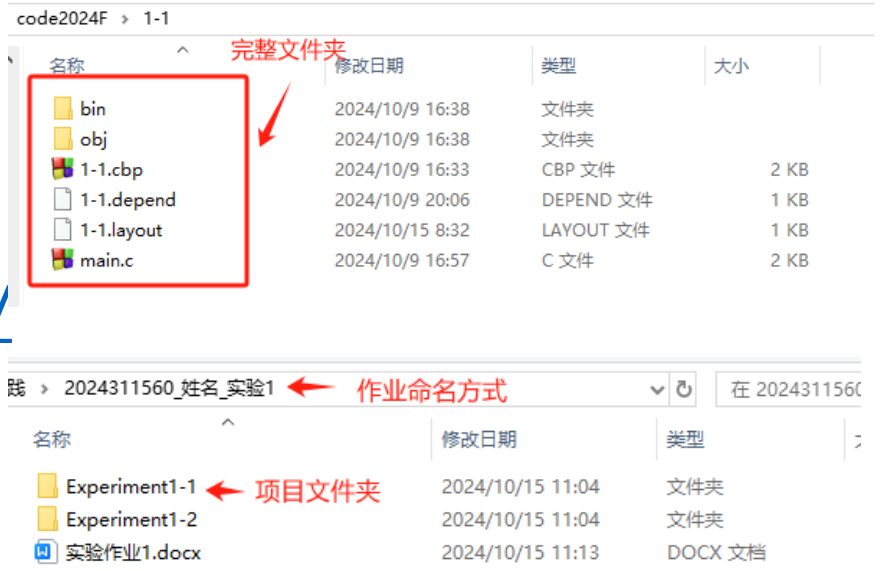
6001 反应速度游戏

- TODO部分函数说明

函数名	功能	输入参数	输出
load_records	从 score.txt 读取历史成绩到数组	rec[]：记录数组（输出） max_count：最大条数	返回读取到的记录条数
show_records	以表格形式在屏幕上显示所有成绩	rec[]：记录数组 count：记录条数	无（在屏幕输出）
add_record	追加一条成绩到数组和文件	rec[]：记录数组（输入/输出） count：记录数指针（输出） ms：本次反应时间	无

实验作业提交

- **截止时间：**具体截止日期参考作业平台发布
- **提交内容：**整个项目文件打成zip包（注意不是.c文件，是完整的工程文件夹）
- **命名要求：**学号_姓名_实验编号
- **HITsz Grader使用指南**
  - 登录网址：： http://grader.tery.top:8001/
  - **初始用户名、密码均为学号，登录后请修改**
  - 作业提交仅支持.zip格式，100MB
  - 作业提交截止时间内，可以重新提交作业，不限次数





请同学们开始实验

GO!
TAKE YOU ON A TRIP

哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

探索 从未停止